

PYTHON COURSE SUMMARY (Till Now)

This document lists all topics covered so far, along with their important methods, parameters, and applications.

1. PRINT FUNCTION

`print(value)`

Parameter:

- value → anything you want to display

Application:

- Display output
 - Debugging
 - Showing results
-

2. VARIABLES

`variable_name = value`

Application:

- Store data
 - Reuse values
 - Make program dynamic
-

3. TYPE FUNCTION

`type(object)`

Parameter:

- object → variable or value

Application:

- Check datatype
 - Debug type errors
-

4. TYPE CASTING

`int(value) float(value) str(value) list(value) tuple(value) set(value)`

Parameter:

- value → convertible object

Application:

- Convert string input to number
 - Remove duplicates (set)
 - Make immutable (tuple)
 - Enable modification (list)
-

5. STRINGS

Methods:

1. `upper()` No parameter Converts text to uppercase
2. `lower()` No parameter Converts text to lowercase
3. `replace(old, new)` old → text to replace new → replacement text Used for cleaning data
4. `split(separator)` separator → character to split by Converts string into list
5. `strip()` No parameter Removes extra spaces

Application:

- Data cleaning
 - Text formatting
 - Input processing
-

6. LIST

Created using: []

Important Methods:

append(value)

- Adds element at end

insert(index, value)

- Inserts at specific position

remove(value)

- Removes first matching value

pop(index)

- Removes element at index
- Default removes last

sort()

- Sorts list ascending

reverse()

- Reverses order

len(list)

- Returns length

max(list), min(list), sum(list)

- Aggregation functions

Application:

- Store dynamic data
 - Data analysis
 - Ordered collections
-

7. TUPLE

Created using: ()

Properties:

- Ordered
- Immutable
- Indexed

Methods:

count(value)

- Returns number of occurrences

index(value)

- Returns first index position

Application:

- Fixed configuration
 - Coordinates
 - Safe data storage
-

8. SET

Created using: {}

Properties:

- Unordered
- Unique values only
- No indexing

Methods:

add(value) remove(value) discard(value) pop() union(other_set) intersection(other_set) difference(other_set)

Application:

- Remove duplicates
- Membership testing
- Mathematical set operations

✓ SESSION 6 — DICTIONARIES & SETS

DICTIONARY

Dictionary stores data in key-value pairs.

Created using {}

Syntax: { key : value }

Keys must be unique. Values can be anything.

Used when:

- You want structured data
- You want mapping
- You want fast lookup

```
# Creating dictionary
```

```
product = {  
    "Laptop": 50000,  
    "Mobile": 20000,  
    "Tablet": 15000  
}
```

```
print(product)
```

```
{'Laptop': 50000, 'Mobile': 20000, 'Tablet': 15000}
```

Important Operations:

1. Access value
 - dictionary[key]
2. Add new key-value
 - dictionary[new_key] = value
3. Update value
 - dictionary[key] = new_value
4. Delete item
 - del dictionary[key]
5. Get all keys
 - dictionary.keys()
6. Get all values
 - dictionary.values()
7. Get key-value pairs
 - dictionary.items()

Start coding or [generate](#) with AI.

```
product = {
    "Laptop": 50000,
    "Mobile": 20000,
    "Tablet": 15000
}

# Access
print(product["Laptop"])

# Add
product["Tablet"] = 15000
print(product)

# Update
product["Mobile"] = 22000
print(product)

# Delete
del product["Laptop"]
print(product)

# Keys
print(product.keys())

# Values
print(product.values())

# Items
print(product.items())

50000
{'Laptop': 50000, 'Mobile': 20000, 'Tablet': 15000}
{'Laptop': 50000, 'Mobile': 22000, 'Tablet': 15000}
{'Mobile': 22000, 'Tablet': 15000}
dict_keys(['Mobile', 'Tablet'])
dict_values([22000, 15000])
dict_items([('Mobile', 22000), ('Tablet', 15000)])
```

Demo: Create product-price dictionary. Then:

- Add a product
- Update price
- Remove product

```
products = {
    "Pen": 10,
    "Notebook": 50
}

products["Pencil"] = 5
products["Pen"] = 12
del products["Notebook"]

print(products)

{'Pen': 12, 'Pencil': 5}
```

```
my_dict = {}

key = input("Enter key: ")
value = input("Enter value: ")

my_dict[key] = value

print(my_dict)

Enter key: 5
Enter value: fog
{'5': 'fog'}
```

✓ SESSION 7 — CONDITIONAL STATEMENTS

Used for decision making.

Syntax:

if condition: statement

elif condition: statement

else: statement

Conditions always evaluate to True or False.

```
# revenue = 120000
revenue = int(input("Enter Revenue: "))

# target = 130000
target = int(input("Enter TARGET: "))
# A = 10000
A = int(input("Enter SALARY: "))

if revenue > target:
    print("Achieved")
    A = A + 2000
    print(A)
# if revenue < target:
# else:
#     print("Not Achieved")
#     A = A - 2000
#     print(A)
```

```
Enter Revenue: 100000
Enter TARGET: 50000
Enter SALARY: 20000
Achieved
22000
```

- if → first condition
- elif → additional condition
- else → fallback condition

Used when multiple possibilities exist.

```
score = int(input("Enter Marks : "))

if score >=33:
    print("Pass")
    print("Pass")
    print("Pass")
    print("Pass")
    print("Pass")
    print("Pass")
    print("Pass")
    print("Pass")
# if score < 33:
#     print("Fail")
else:
    print("Fail")
    print("Fail")
    print("Fail")
    print("Fail")
    print("Fail")
    print("Fail")
    print("Fail")
```

```
Enter Marks : 23
Fail
Fail
Fail
Fail
Fail
Fail
Fail
```

```

score = int(input("Enter Marks : "))

# if score >= 90:
if score >= 50:
    print("Bad")
elif score >= 75:
    print("Good")
# if score >= 60:
#     print("can improve")
elif score >= 90:
    print("Excellent")
else:
    print("poor")

```

Enter Marks : 78
Bad

```

score = int(input("Enter Marks : "))

if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Good")
elif score >= 60:
    print("can improve")
elif score >= 50:
    print("bad")
else:
    print("poor")

```

Enter Marks : 78
Good

```

price = int(input("Enter Marks : "))

if price >= 1000:
    print("payble Amount",price*0.70)
elif price >= 500:
    print("payble Amount",price*0.80)
# elif price >= 60:
#     print("can improve")
elif price >= 200:
    print("payble Amount",price*0.90)
else:
    print("No Discount ",price)

```

Enter Marks : 100
No Discount 100

```

age = int(input("Enter Age : "))

if age >=60:
    print("Senior Citizen")
elif age>=18:
    print("Adult")
else:
    print("Teenager")
else :
    print("child")

```

File ["/tmp/ipykernel_11943/2077905530.py"](#), line 9

```

else :
^
SyntaxError: invalid syntax

```

Next steps: [Explain error](#)

```

age = int(input("Enter Age : "))

if age >= 13:
    if age >=60:
        print("Senior Citizen")
    elif age>=18:
        print("Adult")

```

```

else:
    print("Teenager")
else :
    print("Child")

```

Enter Age : 54
Adult

Nested condition means:

- An if statement inside another if statement.

Used when checking multiple levels.

```

revenue = 120000
target = 100000
region = input("Enter Region: ")

if revenue > target:
    if region == "North":
        print("North Region Achieved Target")
    elif region == "South":
        print("South Region Achieved Target")
    else:
        print("Other Region Achieved Target")
else:
    print("Target Not Achieved")

```

Enter Region: South
South Region Achieved Target

✓ PYTHON MASTER PRACTICE SET

Covers Sessions 1 to 7

Easy → Moderate → Difficult

EASY LEVEL

1. Print the message: "Python Practice Session".
2. Create a variable called age and store your age. Print its datatype.
3. Take user input for marks and convert it into float.
4. Convert the string "data science" into uppercase.
5. From the string "MachineLearning", print the last character.
6. Create a list [10, 20, 30, 40]. Print the second element.
7. Add 50 to the end of the above list.
8. Create a tuple (1, 2, 3, 4). Print the length of the tuple.
9. Create a set with duplicate values: {1,2,2,3,3,4}. Print the set.
10. Create a dictionary with: "Pen": 10 "Notebook": 50 Print the price of "Pen".

MODERATE LEVEL

11. Take input for product price as string. Convert it to float. Add 18% GST and print final price.
12. Given the list: marks = [85, 90, 85, 92, 90] Remove duplicates and sort the result.
13. From the string: " HP-Laptop " Perform:
 - Remove extra spaces
 - Convert to lowercase
 - Replace "-" with space
14. Create a dictionary: { "Laptop": 50000, "Mobile": 20000 }
 - Add "Tablet": 15000

- Update Mobile price to 22000
- Delete Laptop

15. Create two sets: A = {1,2,3,4} B = {3,4,5,6} Find:

- Union
- Intersection
- Difference A - B

16. Take revenue and target as input. If revenue > target print "Achieved" Else print "Not Achieved"

17. Take marks as input. If marks >= 90 print "Excellent" elif marks >= 75 print "Good" else print "Needs Improvement"

DIFFICULT LEVEL

18. Given a list of numbers: [10, 20, 30, 40, 50]

- Convert it to tuple
- Convert tuple to set
- Convert set back to list Print each step.

19. Given: products = { "Pen": 10, "Notebook": 50, "Pencil": 5 }

Calculate total price of all products. If total > 60 print "High Inventory Value" Else print "Normal Inventory Value"

20. Take a sentence as input.

- Convert to lowercase
- Split into words
- Convert words into set Print unique words only.

21. Create a nested condition: revenue = 120000 target = 100000 region = "North"

If revenue > target: If region == "North": Print "North Region Achieved" Else: Print "Other Region Achieved" Else: Print "Target Not Achieved"

22. Given a tuple: (10, 20, 30, 20, 40, 20)

- Count how many times 20 appears.
- Find index of 30.

23. Create a list of 5 numbers from user input. Print:

- Maximum
- Minimum
- Average

24. Given: data = ["Apple", "apple", "APPLE", "Banana"]

Convert all elements to lowercase. Remove duplicates. Print final cleaned list.

25. Create a dictionary where: key = product name value = quantity

If quantity > 100 print "High Stock" Else print "Low Stock"

Start coding or generate with AI.

```
Python Practice Session
<class 'int'>
Enter marks: 67
Converted marks: 67.0
DATA SCIENCE
g
20
[10, 20, 30, 40, 50]
4
{1, 2, 3, 4}
10
Enter product price: 700
Final Price: 826.0
[85, 90, 92]
hp laptop
{'Mobile': 22000, 'Tablet': 15000}
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference A-B: {1, 2}
Enter revenue: 4566
```

```
Enter target: 5000
Not Achieved
Enter marks: 60
Needs Improvement
Tuple: (10, 20, 30, 40, 50)
Set: {40, 10, 50, 20, 30}
List again: [40, 10, 50, 20, 30]
High Inventory Value
Enter sentence: hii i am sahil
Unique words: {'', 'hii', 'i', 'sahil', 'am'}
North Region Achieved
Count of 20: 3
Index of 30: 2
Enter number 1: 5
Enter number 2: 4
Enter number 3: 9
Enter number 4: 10
Enter number 5: 3
Numbers entered: [5, 4, 9, 10, 3]
Maximum: 10
Minimum: 3
Average: 6.2
Cleaned list: ['banana', 'apple']
Pen High Stock
Notebook Low Stock
```

Start coding or [generate](#) with AI.